

HAL 9000 System La Salle URL

Operating Systems

Design and Implementation of a Music Streaming Platform

Marc Escoté & Adrián-Jorge Sánchez

Academic Year 2023-2024

Table of Contents

TABLE OF CONTENTS.....	2
DESIGN.....	3
PROGRAM STRUCTURE	3
□ <i>connections.h / connections.c</i>	3
□ <i>configs.h / configs.c</i>	3
□ <i>functions.h / functions.c</i>	3
□ <i>discovery.c</i>	3
□ <i>bowman.c</i>	3
□ <i>poole.c</i>	3
A) DIAGRAMS	4
<i>Bowman Process</i>	4
<i>Poole Process</i>	7
<i>Discovery Process</i>	10
B) DATA STRUCTURES	12
C) SYSTEM RESOURCES	15
<i>Phase 1</i>	15
<i>Phase 2</i>	15
<i>Phase 3</i>	16
<i>Phase 4</i>	18
<i>Optional Phases</i>	18
OBSERVED PROBLEMS	19
TEMPORAL ESTIMATION.....	20
CONCLUSIONS AND PROPOSALS FOR IMPROVEMENT	21
REFERENCES	22
BIBLIOGRAPHY	22

Design

Program Structure

The project is structured in the following manner:

- **connections.h / connections.c**

Handles socket connections, server setup, and error management. Key functions include configServer, openConnection, checkPort, sendError, readFrame, sendFrame, freeFrame, getFileData, and configQueue.

- **configs.h / configs.c**

Defines data structures and reads configuration files. Key functions include readConfigDis, readConfigBow, readConfigPol, readSongs, and readPlaylists.

- **functions.h / functions.c**

Contains shared functions for the project. Key functions include print, readNum, readLine, readUntil, checkName, capitalize, removeWhiteSpaces, checkCommand, getString, getSongName, and getMd5.

- **Semaphores_v2.h**

Library containing different functions that facilitate the use of semaphores.

- **discovery.c**

Implements the main Discovery server functionality. Key functions include connectionHandler, buildSelect, and main.

- **bowman.c**

Implements the Bowman server functionality. Contains functions specific to Bowman interactions.

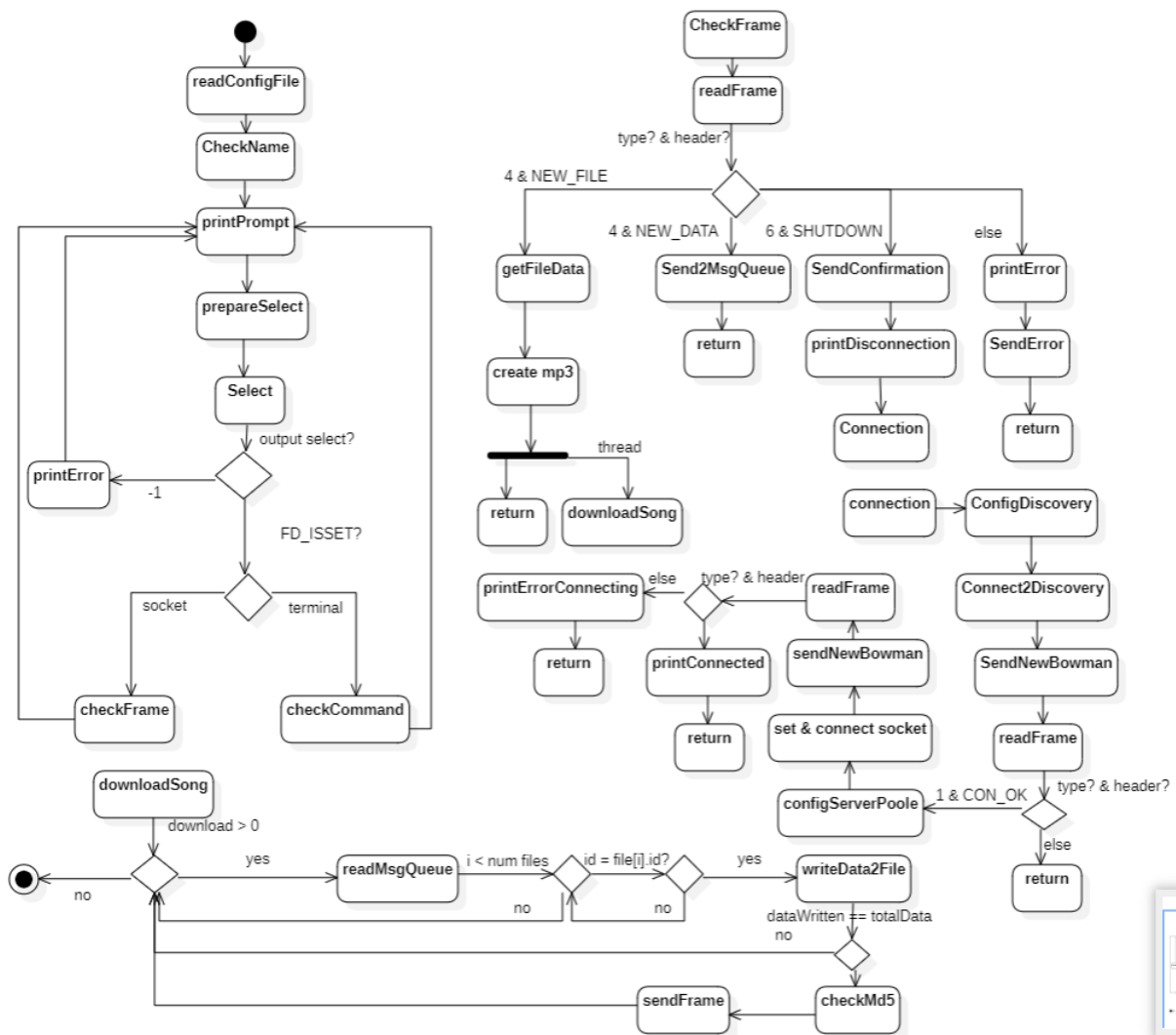
- **poole.c**

Implements the Poole server functionality. Contains functions specific to Poole interactions.

a) Diagrams

Bowman Process

The Bowman process is designed to act as a user interface that connects to a music server which can be from a list of Pooles that are connected to the system. The user can interact with the bowman using the command line to ask for various actions such as list songs, playlists, downloading songs and managing downloads.



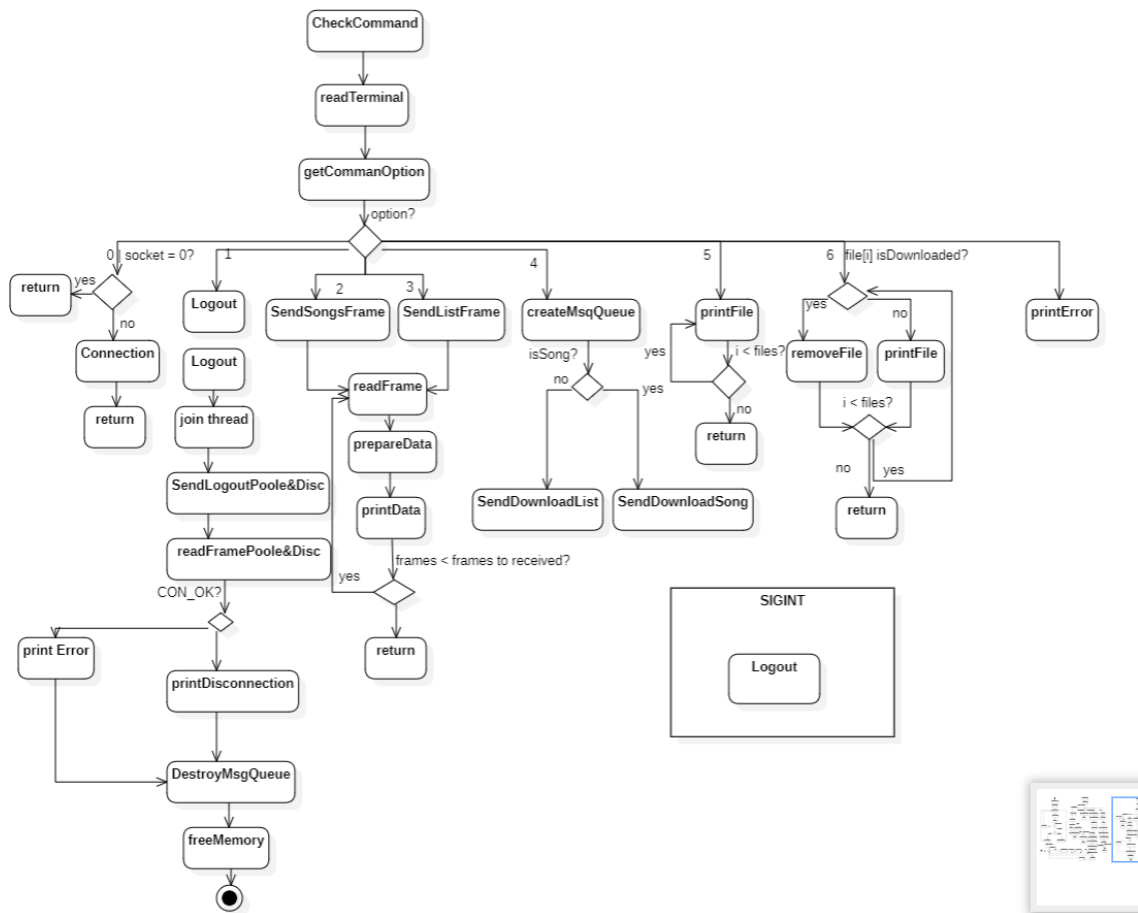
The functionality is as follows:

Bowman starts by reading its configuration file, which contains the essential information like the user's name, directory name, IP and port. This data is crucial for establishing a connection with the Discovery server.

Once the configuration is read the code prepares a select and executes it, this will check the data received from the socket or from the terminal. Once it exits the select checks both file descriptors, if the file descriptor which received something is from the terminal it executes checkCommand function. If the file descriptor is the socket, it executes the checkFrame function.

CheckFrame reads the frame received and checks if type and header:

- If type 4 and its header is NEW_FILE, then it gets from the data of the frame different information regarding the song to be download and stores it, then it creates the mp3 file where it will write the contents it is going to be receiving and then it throw a threads that executes downloadSong. DownloadSong, will check if there are more than 0 downloads, if there are, then it reads the message queue and checks the id of the data received. When it finds the file with the id it received it writes the data received into that songs file. Afterwards, it checks if the data received for that file is all the data to be received. If it is then it checks the md5 of the file created and the md5 received from poole and sends to poole the result. Finally, it returns to the start of the function.
- If type is 4, but the header is newData, then it sends the frame read to the message queue.
- If type is 6 and header is shutdown, then it sends a confirmation, prints the disconnection from the server and closes the socket. Then it executes the connection function. Connection configs the discover server and sets and connects to that socket. Then it sends a frame to inform of its connection and reads a frame to check the server acknowledged its connection and get the data of the poole server. With the data of the poole server, it configs the server, sets and connects to that server and, again, sends a frame informing of the new connection and reads a frame to check the server acknowledged that connection. Finally, it prints a connection message or an error message depending on the frame read.
- If none of the above happens, then it prints an error message.



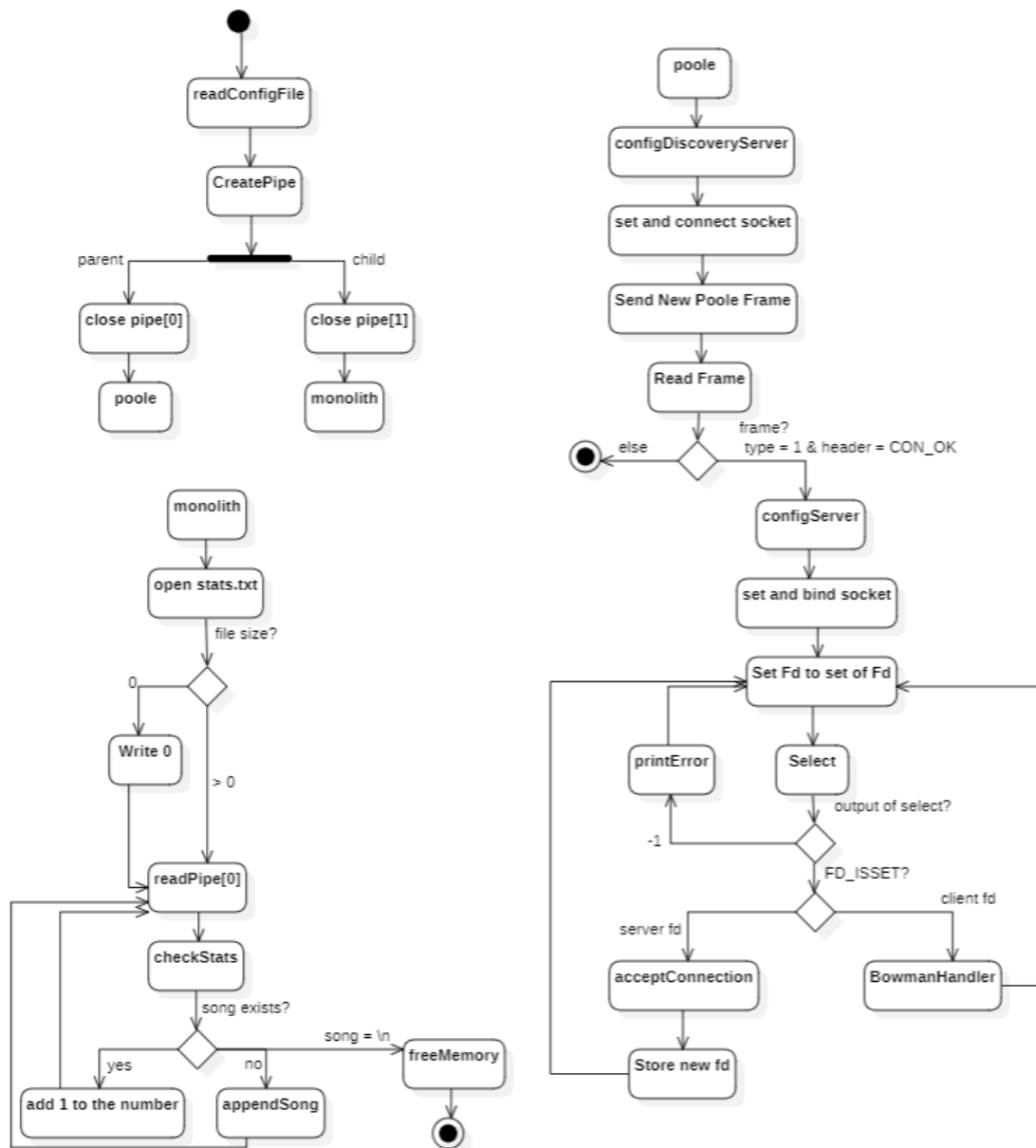
As said before, when a user input is detected, the Bowman process executes the checkCommand. CheckCommand reads the terminal and gets the commandOptions, which is a number depending on the command entered (0 = CONNECT, 1 = LOGOUT, 2 = LIST SONGS...). Then it checks the option:

- If it is 0, then it checks if it is already connected, if it is, then it ends the function. If not, then it executes the Connection function previously explained.
- If it is 1, then it executes the logout function. Which joins the threads, sends frames of logout to discovery and poole and checks its response. Printing an error or not depending on the response. Then it destroys the message queue, frees the memory and ends the program.
- If it is 2, then it sends a frame requesting the songs and reads the frames if there are frames, prepares the data received to be printed and prints the data.
- If it is 3, then it sends a frame requesting the playlists and reads the frames if there are frames, prepares the data received to be printed and prints the data.
- If it is 4, creates a message queue and sends a download playlist or download song frame depending on the argument passed with the download command.
- If it is 5, it goes through all the files stored and prints them.
- If it is 6, it goes through all the files, removing the ones already downloaded and printing the others.
- Else, it prints an error message.

In case of detecting a SIGINT, the bowman gratefully terminates the program. Concurrently, It initiates the logout process previously explained.

Poole Process

The Poole process reads the corresponding configuration file passed as a parameter to then act as a music server that the bowman will be connected to, to request specific actions.

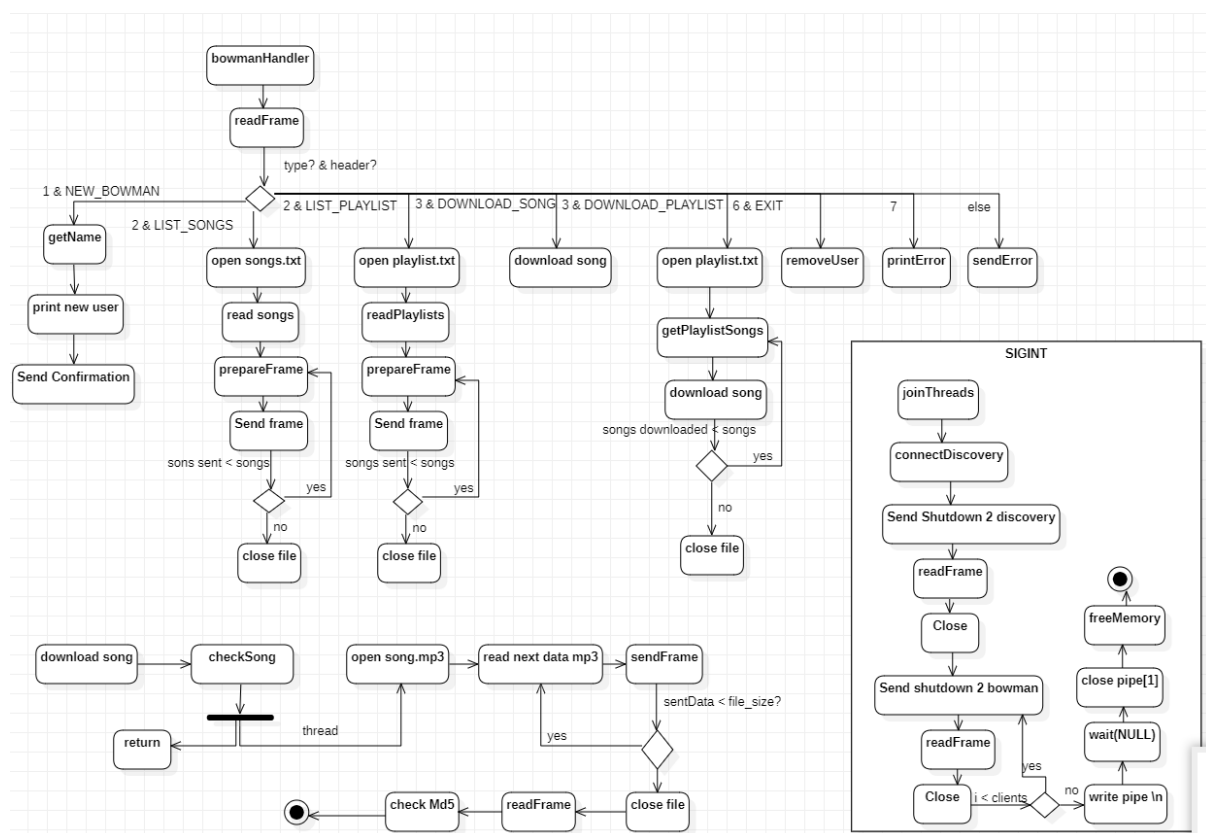


Poole as bowman, starts by reading its configFile, that stores its name, port and ip to open socket and port and ip of discovery server. Once it reads it, it creates a pipe and forks, closing the parent the pipe[0] and executing Poole and closing the child pipe[1] and executing monolith.

Monolith starts by opening the stats.txt and writing a 0 to the file if the file len is 0, file len is git by using lseek to the end of the file. After that it reads the pipe and then goes through the

stats.txt looking for a line that contains the songname it just received through the pipe. If it finds the songs, it updates the number next to the song in the file by adding 1, if it doesn't find the song it adds a new line with the songname and a 1 next to it. It permanently does this until it reads a "\n" through the pipe, in which case it ends the process.

Poole starts by configuring the discovery server with the ip and port read and setting and connecting to the socket. After connecting it sends the frame of NEW_POOLE and reads a frame. If the frame type isn't 1 or header isn't CON_OK the process ends, if not it continues by configuring the Poole server, setting and binding the socket, configuring the number of connections to listen and entering an infinite while loop that adds the server fd and the clients fd to the set of fds and makes a select. If the select return -1 it returns to the start, if not it checks the fds to see which was the one to receive something. If it was the server it accepts the connection and adds the new fd to the array of client fds and returns to the start of the loop. If it is a client fd it executes the bowmanHandler function and returns to the start of the loop.



BowmanHandler, reads one frame from the fd that received something and checks its type and header.

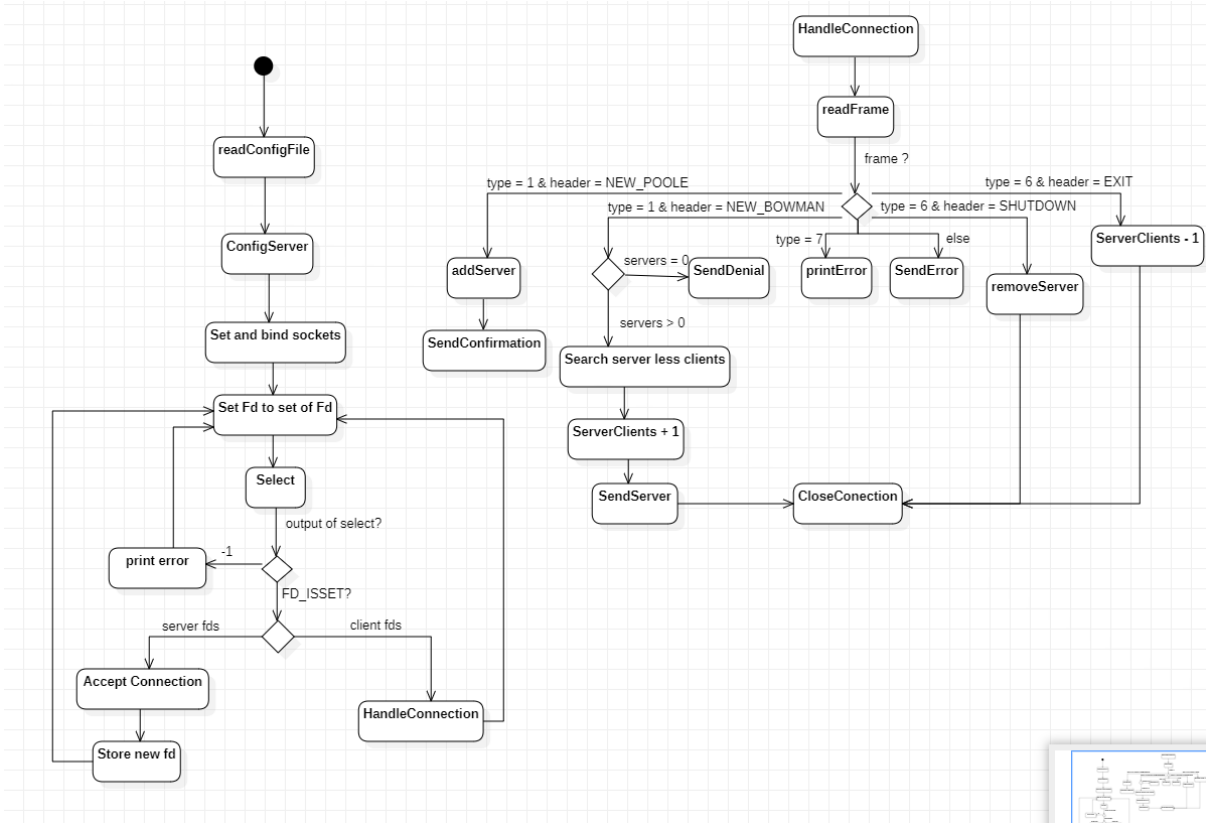
- If its type is 1 and header is NEW_BOWMAN, then it get the name from the data and prints a message informing of the new user and sends a frame confirming it received the frame.
- If its type is 2 and header is LIST_SONGS, then it opens the songs.txt and reads all songs in the file, to then start filling a frame with all the songs that fit in and sends it. If there are more songs to be sent it returns to filling the frame, if not it close the file.

- If the type is 2, but the header is LIST_PLAYLIST, then it opens the playlist.txt and reads all playlists and its songs. As with the songs, it starts filling in a frame all the information that fits in and sends it. If afterwards there is more information to be sent it goes back to filling and sending, if not it closes the file.
- If the type is 3 and the header is DOWNLOAD_SONG, it executes the download song function, which opens the songs.txt file and check whether the song to be download exist within the song.txt file. If it doesn't exists it return, if it exists it throws a new thread that opens the .mp3 of the song to be downloaded reads the amount of data that can fit in the frame and sends a frame, then it checks if the data sent is less than the data to be send. If it is, then it returns to reading and sending, if not it closes the mp3 file and reads a frame, which stores the md5sum of the file received by the client, and check if it is the same as the md5sum of the file it sent.
- If type is 3, but header is DOWNLOAD_PLAYLIST, then it opens the playlist.txt and checks for the playlist to be download and get all its songs. Then it executes the download function previously explained for each song inside the playlist and then closes the file.
- If type is 6 and header is EXIT, then it removes the client from the list of clients.
- If type is 7, then it prints an error.
- If none of the previous is, then it send a frame of type 7.

Finally, if the program receives a SIGINT, then it joins the threads, so it lets them end up sending the songs to be downloaded if there are any. Then it connects to discovery and sends a frame of shutdown, reads a frame and closes the connection. Then for each client connected it sends a frame of shutdown, reads a frame and closes the connection. Then it writes `\n` in the pipe to make the child process end and waits for it to end and then closes the pipe and frees all allocated memory.

Discovery Process

The Discovery process acts as a load balancer, that will open connection for both Poole and Bowman process to connect. Storing the addresses of the poole processes and sending the bowman processes the Poole server with the less amount of clients.



It will start by reading the configuration file, passed as an argument when executing the process, which stores the ip and ports of the socket to be opened for Poole connections and for bowman connections. Then it will configure both Poole and bowman server with the ips and ports read to then set the socket bind it and configure the number of connections to listen.

Once the previous steps have been done, it will enter an infinite loop where it will clear the set of fd and then set the fd of the Poole and bowman server and the fd of the clients connected to then execute a select, this way it can pay attention to different fds without the need of crating threads. When a fd receives something the select will return a value, if the value is -1 then an error will be printed and it will return to the start of the loop. If it is not -1 it will check which fd has received something with the use of the FD_ISSET() function. If the server fd of the Poole or bowman is the one that received something then it will accept the new connection, add the new fd to the array of client fds and return to the start of the loop. If a client fd is the one that received something it will go to the handleConnection() function, which will read the frame from the fd(by reading 256 bytes and splitting the bytes according to the communication protocol) and will check the type and the header of the frame:

- If the frame is of type 1 and its header is “NEW_POOLE”, it will take the name of the server, the ip and the port from the data and add the new server to the list of servers then it will send a confirmation back to the fd.

- If the frame type is 1 but the header is “NEW_BOWMAN”, it will check the number of servers, if the number is 0 it will send a denial message to the fd. If the number is greater than 0, then will go through the list of servers looking for the server with the less amount of clients and add 1 to the amount and add the name of the user to the list of clients of that server. Finally it will send a frame back to the fd containing the name, ip and port of the server and close the connection.
- If the type is 6 and the header is “EXIT”, then it will look for the server storing that user, remove it and subtract one to the amount of clients of that server and close the connection.
- If the type is 6 but the header is “SHUTDOWN”, then it will look for the server that matches the name received in the frame and will remove it from the list of servers and close the connection.
- If the type is 7, then it will print an error message.
- Else it will send a frame of type 7 indicating that the frame received is wrong.

After executing the handleConnection function it will return to the start of the loop and repeat the process.

b) Data Structures

To start with, the first structures used are the structures needed for the config files. Each process has a config file, with data it needs to read and store, so it needs a way to it efficiently.

```
typedef struct {  
    char* server;  
    char* path;  
    char* discovery_ip;  
    int discovery_port;  
    char* user_ip;  
    int user_port;  
} Server_conf;
```

Firstly, `Server_conf` is the structure that Poole uses to store its config file information. It consists of: 4 char pointers, used to dynamically allocate memory, as the length of the data to be stored there is unknown. The name of the server, the path where it has its files stored and the ip of the discovery and user server. Finally, it has 2 integers to store the port of the discovery and user socket.

```
typedef struct {  
    char* user;  
    char* files_path;  
    char* ip;  
    int port;  
} User_conf;
```

Continuing with `User_conf`, which is the structure used by Bowman to store its configuration file information. In this case: 1 char pointer for the name of the user, 1 for the path to where the files will be stored and 1 for the ip of the discovery server to which it will connect, as all of them are of unknown size. Finally, an integer for the port of that discovery server.

```
typedef struct {  
    char* ip_poole;  
    int port_poole;  
    char* ip_bow;  
    int port_bow;  
} Disc_conf;
```

Next is `Disc_conf`, which is the structure used by Discovery to store the information of its configuration file. This structure consists of 2 char pointers, one to store the ip of the socket to be opened for pooles to connect and the other to store the ip of the socket to be opened for bowmans to connect. Finally, 2 integers, one for the port of the socket for pooles and the other for the port of the socket for bowmans.

```
typedef struct {  
    int num_songs;  
    char* name;  
    char** songs;  
} Playlist;
```

Finally, there is Playlist, which is the structure to store the information of the playlist.txt. This one consists of a integer to store the number of songs the playlist has, a char pointer for the name of the playlist and a pointer to a char pointer to store the name of all the songs the playlist has.

Then we have the other set of structures, which are the ones related to connections.

```
typedef struct {  
    char* name;  
    int num_users;  
    char* ip;  
    int port;  
} Server;
```

Starting with Server, a structure used by discovery to store all the poole servers that are running. To check them when a bowman ask for a server and send him the one less populated. This structure has a char pointer for the name of the server, an integer to store the number of users already connected and lastly a char pointer and an integer to store the ip and port of that server respectively.

```
typedef struct {  
    char type;  
    char length[3];  
    char* header;  
    char* data;  
} Frame;
```

Then we have the frame structure, probably the most used structure, used by bowman, poole and discovery to store the data read. As different data is sent as a string of 256 bytes. So later on, when read, it has to be splited into the fields stored in that structure. 1 char for the type of the frame, which is a number between 1 and 7, an array of 3 characters used to store the length, static memory has been used as we know it always consist of 2 number, but it is of size 3 to add the \0 to transform it into an integer using atoi when needed. Then we have 2 char pointers, one for the header and another for the data which are of unknown size.

```
typedef struct {  
    char* list;  
    char* file_name;  
    int file_size;  
    char* md5;  
    int id;  
    int data_received;  
    int fd;  
} File;
```

The File structure is the structure used by bowman to receive the data from a song file. It consists of a char pointer used to store the name of the file, which will be used to create the mp3 file, an integer to store the size of the file and an integer to store the amount of data received until that moment which will be used to know when a file has already received all the data. A char pointer is used to store the md5sum of the file to be received to later on compare it with the actual md5. An integer to store the id of the file which is sued to recognize each frame of data to which file it belongs and an integer to store the file descriptor of the file where the received data is being written.

```
typedef struct {  
    char* name;  
    int fd_pos;  
    int thread_pos;  
} Send;
```

The Send structure is used to store data that the threads that are in charge of sending the data of the files to be download need. As there are more than 1 thing, a structure must be done to store all the things there and send the structure to the thread. It has a char pointer that stores the name of the file and 2 integers, 1 representing the position of the thread which is the position of the id in an array of integers of the song the thread is in charge of. The other is the position of the file descriptor to which the song must be send.

```
typedef struct {  
    long mtype;  
    char data[244];  
} Msg;
```

Lastly, Msg is the structure used by bowman in the message queues. It has a long which stores the type of the message and the other is a static array of characters of 244. It is static because the data to be sent is always 244, which is 256(the size of the frame) minus the type, the length of the header and the header.

c) System Resources

Phase 1

The first phase of the project consists on processing the configuration files for the Poole server and Bowman users. The configuration files contain the essential information such as the executing user's name, the directory where the files will be located, and the IP and port of that will be used to interconnect among them and with the Discovery server implemented in phase2.

System Resources

- File descriptions have been used to open, read and close the configuration file of both Bowman and poole and to write and read from the terminal.
- Signal handling is implemented in the Bowman process to manage and control program interruptions when receiving a SIGINT.

Phase 2

This phase begins initializing the Discovery server. First it reads its configuration file passed as a parameter and sets up sockets to listen for incoming connections from the Poole and Bowman users.

When the Poole process is run, it connects to the discovery server, registering itself. Once connected, the discovery will manage and distribute the new Bowman connections to the different Poole Servers. When the Poole is connected to the Discovery it is acting as a client but then when it disconnects and a Bowman connects to it starts acting as supposed, as a music server.

Similarly, the Bowman process will connect to the Discovery server using the IP and port read from the corresponding configuration file. The bowman is registered in the Discovery server which acts as a manager to tell each bowman to which Poole server he must connect to.

Once the connections are established, the Bowman users will be able to ask for the following functionalities.

List Songs: Bowman listens for user requests, and upon receiving a "List Songs" command, it communicates with the connected Poole server to retrieve and display the available songs.

List Playlists: The "List Playlists" will retrieve and present the existing playlists.

Logout: When the user decides to log out, the Bowman communicates with the Discovery server to inform about the disconnection, and the associated Poole server updates its user count accordingly

System Resources

- In the Bowman, Poole and discovery process of Phase 2, sockets were implemented as a fundamental communication tool to establish the connection between them. The choice of sockets is driven by their suitability for network communication between processes. It is also important to mention that with sockets it is possible to connect a process from the matagalls server with another one started from the Montserrat server.
- For both Discovery and Poole, the select function was used to efficiently manage multiple socket connections concurrently. The use of select allows the server to wait for connections to happen on multiple sockets. This feature ensures responsiveness without the need for excessive polling or threading. This solution could have some scalability problems if the server needs to handle large number of connections simultaneously.
- Signal handling is implemented in the Poole process to manage and control program interruptions when receiving a SIGINT.
- Semaphores have been implemented to protect the stats.txt so that only one poole can access the file at a time. Preventing errors in the reading and writing of the file when multiple poole servers exist.

Phase 3

In the third Phase, the HAL9000 system introduces new functionalities such as downloading a song or a playlist and checking and clearing the previously downloaded files.

File downloading: The system has the option to perform the `DOWNLOAD SONG_NAME` and `DOWNLOAD PLAYLIST_NAME` commands, enabling the Bowman users to request specific songs from the Poole servers that they are connected to. The system also is built to manage when users request non existing songs or playlists.

Download Tracking: The `CHECK DOWNLOADS` command allows Bowman users to view the progress of ongoing downloads with a progress bar that indicates the status of each download. The `CLEAR DOWNLOADS` command enables users to clear completed downloads from the download list.

File Overwriting: The system handles cases where a Bowman receives a file with the same name as a previously received file, ensuring proper overwriting.

Controlled Crashes: The system is designed to control crashes from the Poole servers when `Control + C` is pressed. When a Poole server shuts down or loses its connection, the system detects it and updates how the different users will connect.

System Resources

- Sockets continue to play a crucial role in Phase 3. They facilitate the communication between Bowman and Poole processes. The established socket connections are utilized for the exchange of frames and files between the different processes.
- Threads are used in the Poole server so that when it receives a request for downloading a file, a separate thread will be created to read and send the data. This method ensures that other operations such as managing other user requests at the same time will still work. Bowman uses threads as well, in his case just 1, that is constantly receiving the data of the files to be downloaded through a message queue.
- The Poole and Bowman processes utilize a fork to create a child process that will be in charge of executing the md5sum. This fork is necessary because when using the *execlp* function to execute the md5sum, the process finishes running the *execlp* dies.
- In this Phase, a pipe was used for inter-process communication within the Poole and within the Bowman. In this particular case, a pipe was used to establish a communication method between the Poole/Bowman process and the md5sum process that was previously duplicated using a fork. This communication is crucial to pass the md5sum calculation to the Poole/Bowman process after the fork process executes the command and dies.
- The use of message queues was used for facilitating communication between threads. In this particular case, it plays a crucial role for coordinating the download process and ensuring effective interaction between the threads managing the downloading process and the main procedure. The function is as follows: When the Bowman reads a frame, for example, in list songs, it checks whether the frame received is the intended one (frame of type 2 and header LIST_SONGS) or is one from a download(type 3), in which case it executes a function that sends the frame through message queues to the threads in charge of managing the downloads, which is constantly reading the message queue while there are files downloading. Once done, it reads again until it reads the intended frame or a frame of type 7.
- Mutex was utilized as a mutual exclusion mechanism and control access to shared memory resources during operations that accessed global variables, the sending and receiving of frames or involved the terminal, such as prints or the md5sum (which redirects the terminal file descriptor to the pipe, so no write to the terminal should be do apart from the one of the md5sum). By using mutex, it ensures that only one process will be able to perform or execute that specific operation. This method prevents potential conflicts that might arise when multiple process are running and access/modify the same data.

Phase 4

In this phase, a new process called Monolith was introduced. This process is responsible for computing a series of statistics based on what downloads are requested to the Poole servers. Communication between the Poole and Monolith processes could not be done using sockets or message queues.

System Resources

- The monolith function performs file operations to open read and write the stats.txt file. The standard I/O open, read, write and lseek were used.
- The code is designed to create a pipe (poole2mono) which is intended to be used for communicating between the Poole and the Monolith processes.
- The use of a fork was used to create a child process that is in charge of executing the Monolith process.

Optional Phases

The system detects when a Poole server goes offline due to a SIGINT and updates the load balancer accordingly, as stated. But, additionally, it handles Poole server crashes by reconnecting all Bowmans that were connected to the crashed server, asking the Discovery for the new connection information.

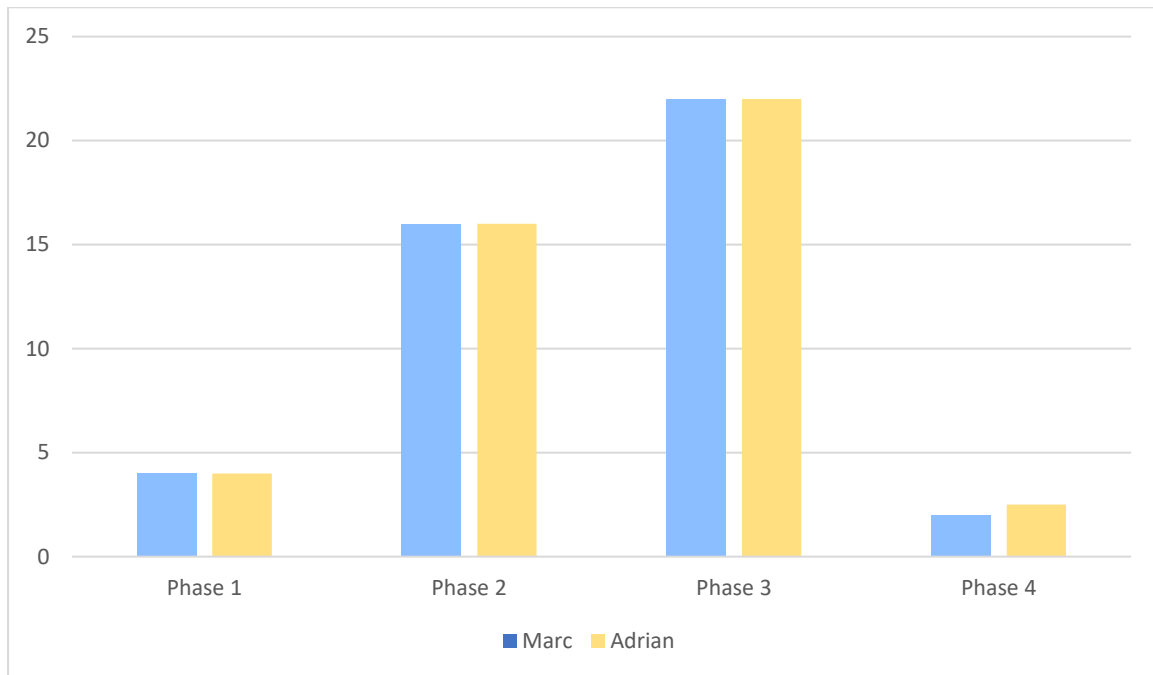
Observed problems

During the development of the HAL 9000 system project, we encountered several challenges that required careful consideration and attention in order to solve.

The main challenge came from the implementation of the select mechanism used for monitoring the connection of multiple Pooles and Bowmans to the Discovery server. This error made the system only be able to connect a single Poole and single Bowman to the Server. In order to solve the issue, we had to carefully modify some pointer parameters that were being passed to some functions that had select-related functionalities. This problem involved debugging and adjusting the way we passed and manipulated pointers to prevent interference and data corruption during the execution of the select.

Another challenge manifested during attempts to download songs. Some issues appeared when utilizing `strlen()` or similar c functions during the reading of received frames, resulting in disruptive bugs. To resolve this, specific sections of the code had to be modified, enhancing its robustness and eliminating bugs associated with the use of string-related functions in the song download process.

Temporal estimation



The following graph shows an estimation of the hours spent for each phase. As seen above, the 2nd and 3rd phase were the longest to develop to the high complexity of the functionalities.

Conclusions and proposals for improvement

The HAL 9000 System, has undergone a significant development and refinement through its various phases of the development. The design and implementation showcase a robust system architecture that incorporate mechanisms such as socket-base communication between processes, thread management and inter-processing communications with tools like pipes or shared memory. As the system progressed from the basic implementation that started in the first phase, new features such as downloading files and keeping track of downloaded files were carried out.

The effective utilization of sockets has been an indispensable step to ensure a seamless communication between the Bowman and Poole processes. The incorporation of threads, particularly in the Poole server, has proven that it is crucial for handling concurrent user requests. The use of forks and pipes has facilitated inter-process communications in sections such as performing the md5sum, this mechanism allowed the execution of background tasks that ensured the proper functioning of certain functionalities. Message queues have streamlined the communication between threads, more in concrete, when coordinating the downloading of files.

Additionally, mutex has played a crucial role in order to ensure the integrity of shared resources, preventing conflicts during simultaneous accesses. This has been particularly important when working with global variables file operations and terminal interactions. The monolith process, which was the last feature introduced, adds a layer of sophistication by computing statistics based on what requests the Bowman users have previously done.

Throughout the development of the project, the design and implementation of the different phases has enhanced the knowledge we had in this field. This project has served as a practical application of all the different topics that we have been learning during this first half of the year. From the foundational understanding of operating systems to the complexity of inter-process communication, socket programming, thread management, and advanced synchronization mechanisms, the project has provided a hands-on experience that helped us learn how machines really work. Overall, the project has been a transformative experience, helping us enrich our academic knowledge by providing a hands on experience of lots of new concepts.

Some new proposals and improvements could include the following:

User Interface Refinement:

Enhance the user interface to provide a more user-friendly experience. This may include implementing a graphical user interface (GUI) or refining command-line interactions to improve user engagement.

Security Measures:

Implement additional security measures, such as encryption for data transmission and robust user authentication, to ensure the integrity and confidentiality of user interactions within the system.

References

This year statement theme has been the famous film: “2001: A Space Odyssey”.

Firstly, the title of the project “HAL 9000 System” takes after the supercomputer that managed the spaceship that appears on that film, which has the same name. Then, “Discovery” references the spaceship already mentioned, which is called **Discovery** One. Bowman and Poole come from the surnames of the crewmen of the spaceship: David **Bowman** and Frank **Poole**. Finally, “Monolith” references the monoliths that appears on the film and around which the film revolves.

Bibliography

Execute file - Linux manual page. (n.d.). <https://linux.die.net/man/3/execlp>

Programació en UNIX per a pràctiques de Sistemes Operatius. (2014).
<https://users.salle.url.edu/~xavier.canaleta/so/assig/LlibreUNIX.pdf>

Select - Linux manual page. (n.d.). <https://man7.org/linux/man-pages/man2/select.2.html>

Getting a file's size in C with lseek? (n.d.). Stack Overflow.
<https://stackoverflow.com/questions/23306552/getting-a-files-size-in-c-with-lseek>

GeeksforGeeks. (2023, October 19). Socket programming in C C.
<https://www.geeksforgeeks.org/socket-programming-cc/>

Open 3 - Linux manual page. (n.d.). <https://linux.die.net/man/3/open>